



UNIVERSIDAD NACIONAL AUTÓNOMA DE
MÉXICO

FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA
e INTERACCIÓN HUMANO COMPUTADORA



PRACTICA 6

NOMBRE COMPLETO: Mora Ortega Judith

Nº de Cuenta: 318006538

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 02

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: Mayo/2026

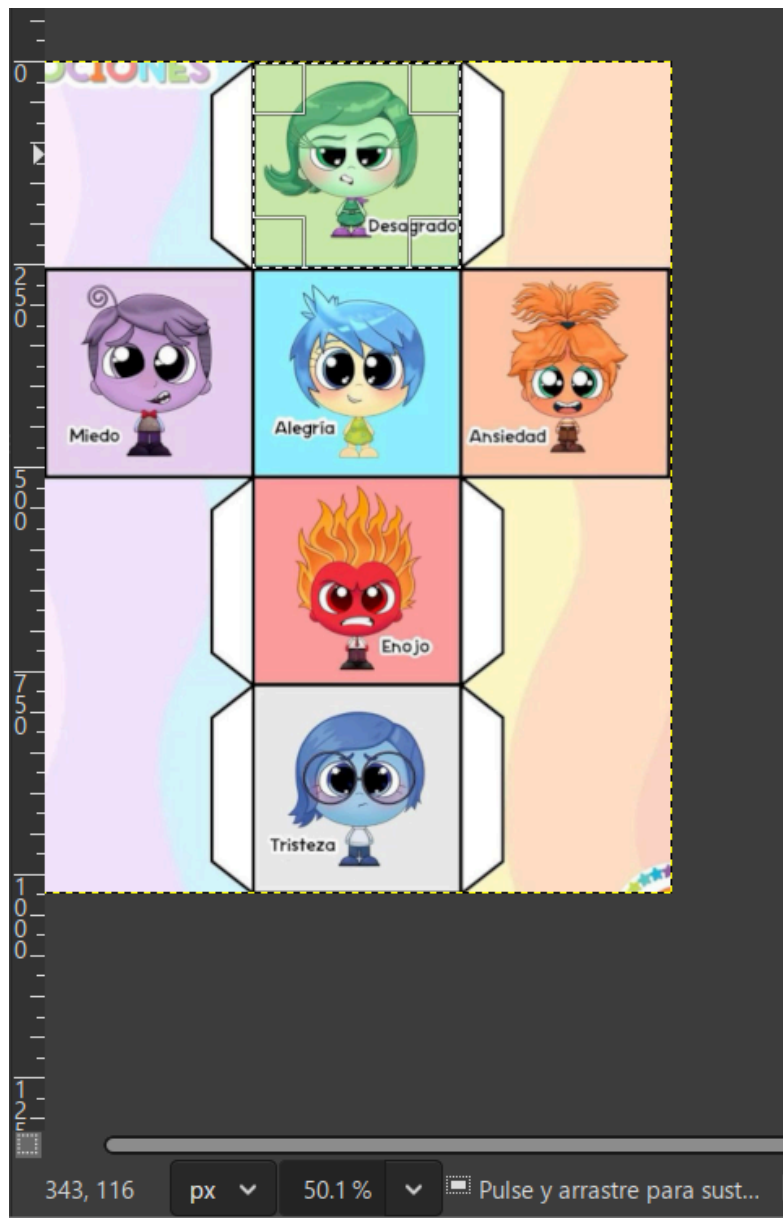
CALIFICACIÓN: _____

Desarrollo de práctica

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Primero realicé el ejercicio de clase que fue texturizar el cubo, una imagen de las emociones.

1. Primero la pasamos GIMP para escalarla y le importó en png.



2. Como lo pide el profesor tenemos que hacerlo en OpenGL. Dando las coordenadas. Una por una haciendo los cálculos que me tarde una eternidad.

Declaramos todo lo que vamos a ocupar

```
Texture dadoTexture;
```

```
// — Texturas —————  
pisoTexture = Texture("Textures/piso.tga");        pisoTexture.LoadTextureA();  
dadoTexture = Texture("Textures/Dado_emociones.png"); dadoTexture.LoadTextureA();
```

```
// — DADO 6 CARAS —————  
color = glm::vec3(1.0f, 1.0f, 1.0f);  
model = glm::mat4(1.0f);  
model = glm::translate(model, glm::vec3(-1.5f, 4.5f, -2.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
glUniform3fv(uniformColor, 1, glm::value_ptr(color));  
dadoTexture.UseTexture();  
meshList[4]->RenderMesh();
```

Una vez vamos a mapear todo los vértices del cubo, con la textura

```
// —————  
// Dado de 6 caras  
// —————  
void CrearDado()  
{  
    unsigned int cubo_indices[] = {  
        0,1,2, 2,3,0,        // front  
        8,9,10, 10,11,8,     // back  
        12,13,14, 14,15,12,  // left  
        16,17,18, 18,19,16,  // bottom  
        20,21,22, 22,23,20,  // top  
        4,5,6, 6,7,4        // right  
    };  
}
```

```

GLfloat cubo_vertices[] = {
    // front
    -0.5f,-0.5f, 0.5f, 0.3346f,0.0020f, 0.0f,0.0f,-1.0f,
    0.5f,-0.5f, 0.5f, 0.6653f,0.0020f, 0.0f,0.0f,-1.0f,
    0.5f, 0.5f, 0.5f, 0.6653f,0.2490f, 0.0f,0.0f,-1.0f,
    -0.5f, 0.5f, 0.5f, 0.3346f,0.2490f, 0.0f,0.0f,-1.0f,
    // right
    0.5f,-0.5f, 0.5f, 0.3423f,0.7520f, -1.0f,0.0f,0.0f,
    0.5f,-0.5f,-0.5f, 0.6615f,0.7520f, -1.0f,0.0f,0.0f,
    0.5f, 0.5f,-0.5f, 0.6615f,0.9912f, -1.0f,0.0f,0.0f,
    0.5f, 0.5f, 0.5f, 0.3423f,0.9912f, -1.0f,0.0f,0.0f,
    // back
    -0.5f,-0.5f,-0.5f, 0.3372f,0.2539f, 0.0f,0.0f,1.0f,
    0.5f,-0.5f,-0.5f, 0.6627f,0.2539f, 0.0f,0.0f,1.0f,
    0.5f, 0.5f,-0.5f, 0.6627f,0.5029f, 0.0f,0.0f,1.0f,
    -0.5f, 0.5f,-0.5f, 0.3372f,0.5029f, 0.0f,0.0f,1.0f,
    // left
    -0.5f,-0.5f,-0.5f, 0.3346f,0.5029f, 1.0f,0.0f,0.0f,
    -0.5f,-0.5f, 0.5f, 0.6627f,0.5029f, 1.0f,0.0f,0.0f,
    -0.5f, 0.5f, 0.5f, 0.6627f,0.7520f, 1.0f,0.0f,0.0f,
    -0.5f, 0.5f,-0.5f, 0.3346f,0.7520f, 1.0f,0.0f,0.0f,
    // bottom
    -0.5f,-0.5f, 0.5f, 0.0090f,0.5029f, 0.0f,1.0f,0.0f,
    0.5f,-0.5f, 0.5f, 0.3346f,0.5029f, 0.0f,1.0f,0.0f,
    0.5f,-0.5f,-0.5f, 0.3346f,0.7520f, 0.0f,1.0f,0.0f,
    -0.5f,-0.5f,-0.5f, 0.0090f,0.7520f, 0.0f,1.0f,0.0f,
    // top
    -0.5f,0.5f, 0.5f, 0.6691f,0.5029f, 0.0f,-1.0f,0.0f,
    0.5f,0.5f, 0.5f, 0.9910f,0.5029f, 0.0f,-1.0f,0.0f,
    0.5f,0.5f,-0.5f, 0.9910f,0.7520f, 0.0f,-1.0f,0.0f,
    -0.5f,0.5f,-0.5f, 0.6691f,0.7520f, 0.0f,-1.0f,0.0f,
};

```

**Lo importante es lo que está en la segunda columna.

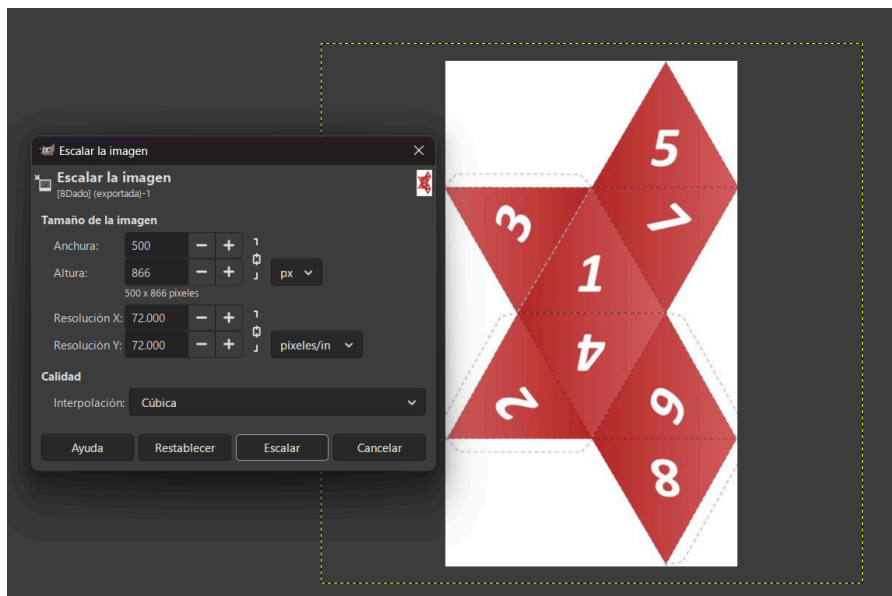
Ejecución del cubo



En la segunda parte de la entrega de esta es hacer un octaedro, (Dado el cual tener números [1 al 8])

Se hicieron los pasos anteriormente mencionados.

1. Imagen que vamos a ocupar para la textura del cubo



2. Vamos a mapear y declarar todos funciones.

```
Texture dado8Texture;
```

```
dado8Texture = Texture("Textures/Dado_emojies.png"); dado8Texture.LoadTexture();  
dado8Texture = Texture("Textures/8Dado.png");      dado8Texture.LoadTextureA();
```

```
// — DADO 8 CARAS —  
model = glm::mat4(1.0f);  
model = glm::translate(model, glm::vec3(1.5f, 4.5f, -2.0f));  
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));  
dado8Texture.UseTexture();  
meshList[5]->RenderMesh();
```

El mapeo

```
// —————  
// Dado de 8 caras (octaedro) |  
// —————  
void CrearDado8()  
{  
    GLfloat oct_vertices[] = {  
        // Cara 1  
        0.0f, 1.0f, 0.0f, 0.7540f,0.0000f, 0.0f, 0.5f, 0.5f,  
        -1.0f, 0.0f, 0.0f, 0.5000f,0.2483f, 0.0f, 0.5f, 0.5f,  
        0.0f, 0.0f, 1.0f, 1.0000f,0.2483f, 0.0f, 0.5f, 0.5f,  
        // Cara 2  
        0.0f,-1.0f, 0.0f, 0.7600f,0.4965f, 0.0f,-0.5f, 0.5f,  
        0.0f, 0.0f, 1.0f, 0.5000f,0.2483f, 0.0f,-0.5f, 0.5f,  
        -1.0f, 0.0f, 0.0f, 1.0000f,0.2483f, 0.0f,-0.5f, 0.5f,  
        // Cara 3  
        0.0f, 1.0f, 0.0f, 0.7600f,0.4965f, 0.5f, 0.5f, 0.0f,  
        0.0f, 0.0f, 1.0f, 0.5000f,0.7448f, 0.5f, 0.5f, 0.0f,  
        1.0f, 0.0f, 0.0f, 1.0000f,0.7448f, 0.5f, 0.5f, 0.0f,  
        // Cara 4  
        0.0f,-1.0f, 0.0f, 0.7520f,0.9931f, 0.5f,-0.5f, 0.0f,  
        1.0f, 0.0f, 0.0f, 0.5000f,0.7448f, 0.5f,-0.5f, 0.0f,  
        0.0f, 0.0f, 1.0f, 1.0000f,0.7448f, 0.5f,-0.5f, 0.0f,  
        // Cara 5  
        0.0f, 1.0f, 0.0f, 0.0000f,0.2483f, 0.0f, 0.5f,-0.5f,  
        1.0f, 0.0f, 0.0f, 0.2400f,0.4965f, 0.0f, 0.5f,-0.5f,  
        0.0f, 0.0f,-1.0f, 0.5000f,0.2483f, 0.0f, 0.5f,-0.5f,  
    }  
}
```

```

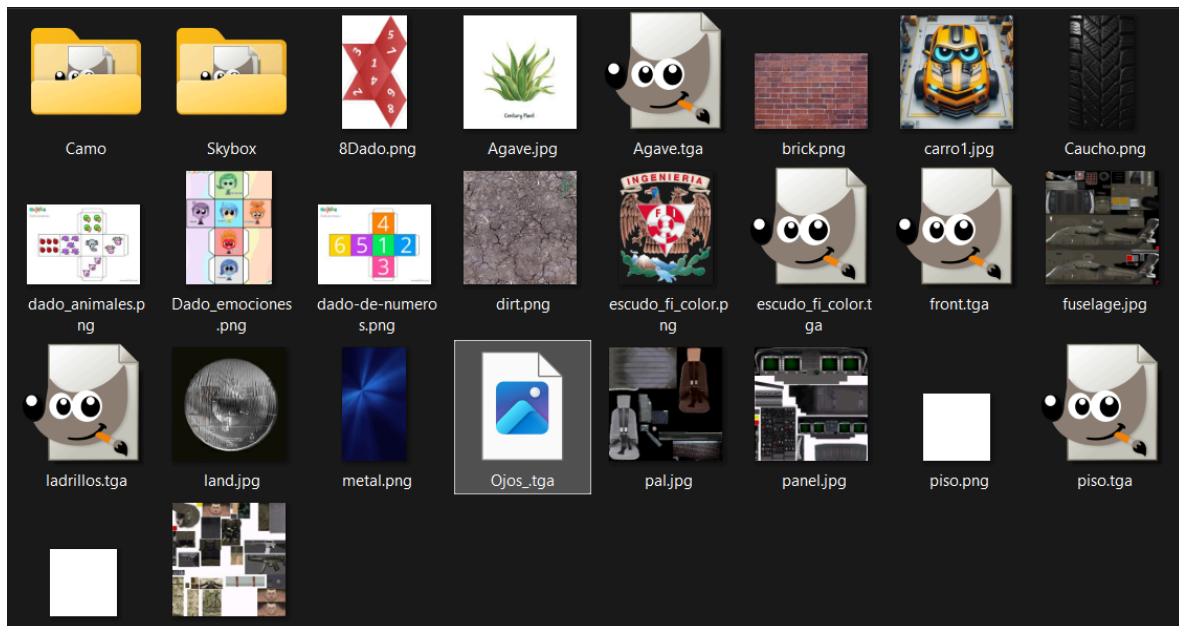
// Cara 5
0.0f, 1.0f, 0.0f, 0.0000f,0.2483f, 0.0f, 0.5f,-0.5f,
1.0f, 0.0f, 0.0f, 0.2400f,0.4965f, 0.0f, 0.5f,-0.5f,
0.0f, 0.0f,-1.0f, 0.5000f,0.2483f, 0.0f, 0.5f,-0.5f,
// Cara 6
0.0f,-1.0f, 0.0f, 0.5000f,0.2483f, 0.0f,-0.5f,-0.5f,
0.0f, 0.0f,-1.0f, 0.7540f,0.4965f, 0.0f,-0.5f,-0.5f,
1.0f, 0.0f, 0.0f, 0.2400f,0.4965f, 0.0f,-0.5f,-0.5f,
// Cara 7
0.0f, 1.0f, 0.0f, 0.5000f,0.7448f, -0.5f, 0.5f, 0.0f,
0.0f, 0.0f,-1.0f, 0.2400f,0.4965f, -0.5f, 0.5f, 0.0f,
-1.0f, 0.0f, 0.0f, 0.7540f,0.4965f, -0.5f, 0.5f, 0.0f,
// Cara 8
0.0f,-1.0f, 0.0f, 0.0000f,0.7448f, -0.5f,-0.5f, 0.0f,
-1.0f, 0.0f, 0.0f, 0.5000f,0.7448f, -0.5f,-0.5f, 0.0f,
0.0f, 0.0f,-1.0f, 0.2400f,0.4965f, -0.5f,-0.5f, 0.0f,
};
unsigned int oct_indices[] = {
    0,1,2, 3,4,5, 6,7,8, 9,10,11,
    12,13,14, 15,16,17, 18,19,20, 21,22,23
};
Mesh* dado8 = new Mesh();
dado8->CreateMesh(oct_vertices, oct_indices, 192, 24);
meshList.push_back(dado8); // [5]
}

```

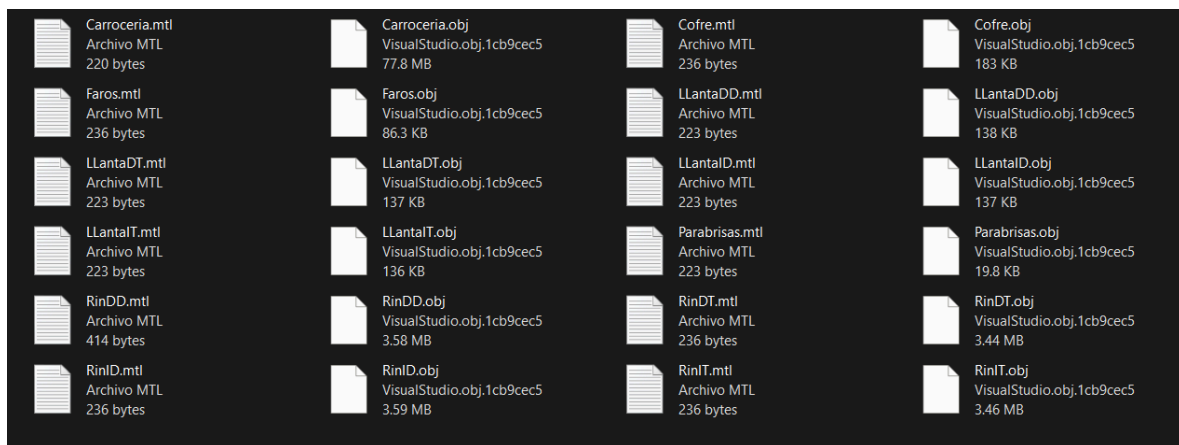
Ejecución del Dado Octaedro



Ejercicio 2 :Texturizar las llantas y los rines con un material adecuado.



Las partes que se modificaron fueron varias empezando por las carpeta de modelos y texturas.



Volvemos a cargar los nuevos modelos.

Pero aquí dividí el carro más partes declaración lo que vamos a usar, la texturizar correctamente.


```

Texture cauchoTexture; // para llantas (LlantaDD, LlantaDT, LlantaID, LlantaIT)
Texture metalTexture;  // para rines (RinDD, RinDT, RinID, RinIT)
Texture ojosTexture;   // para parabrisas

// Coche - carrocería y extras
Model Carroceria_M;
Model Cofre_M;
Model Parabrisas_M;
Model Faros_M;

// Llantas (caucho)
Model LlantaDD_M; // Delantera Derecha
Model LlantaDT_M; // Trasera Derecha
Model LlantaID_M; // Delantera Izquierda
Model LlantaIT_M; // Trasera Izquierda

// Rines (metal)
Model RinDD_M;
Model RinDT_M;
Model RinID_M;
Model RinIT_M;

```

```

// — Texturas —————
pisoTexture = Texture("Textures/piso.tga");           pisoTexture.LoadTextureA();
dadoTexture = Texture("Textures/Dado_emociones.png"); dadoTexture.LoadTextureA();
dado8Texture = Texture("Textures/8Dado.png");         dado8Texture.LoadTextureA();
cauchoTexture = Texture("Textures/Caucho.png");       cauchoTexture.LoadTextureA();
metalTexture = Texture("Textures/metal.png");         metalTexture.LoadTextureA();
ojosTexture = Texture("Textures/Ojos_.tga");          ojosTexture.LoadTextureA();

// — Modelos coche (tu auto) —————
Carroceria_M = Model(); Carroceria_M.LoadModel("Models/Carroceria.obj");
Cofre_M = Model(); Cofre_M.LoadModel("Models/Cofre.obj");
Parabrisas_M = Model(); Parabrisas_M.LoadModel("Models/Parabrisas2.obj");
Faros_M = Model(); Faros_M.LoadModel("Models/Faros.obj");

LlantaDD_M = Model(); LlantaDD_M.LoadModel("Models/LlantaDD.obj");
LlantaDT_M = Model(); LlantaDT_M.LoadModel("Models/LlantaDT.obj");
LlantaID_M = Model(); LlantaID_M.LoadModel("Models/LlantaID.obj");
LlantaIT_M = Model(); LlantaIT_M.LoadModel("Models/LlantaIT.obj");

RinDD_M = Model(); RinDD_M.LoadModel("Models/RinDD.obj");
RinDT_M = Model(); RinDT_M.LoadModel("Models/RinDT.obj");
RinID_M = Model(); RinID_M.LoadModel("Models/RinID.obj");
RinIT_M = Model(); RinIT_M.LoadModel("Models/RinIT.obj");

Parabrisas_M = Model();
Parabrisas_M.LoadModel("Models/Parabrisas.obj");
printf("Parabrisas cargado\n");

```

```

// — CARROCERÍA (raíz) —————
color = glm::vec3(0.0f, 0.3f, 0.8f); // azul
model = glm::mat4(1.0f);
model = glm::translate(model, carPosition);
model = glm::scale(model, glm::vec3(0.3f, 0.3f, 0.3f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
Carroceria_M.RenderModel();

glm::mat4 carBase = model; // padre jerárquico

// — PARABRISAS —————
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
modelaux = carBase;
modelaux = glm::translate(modelaux, glm::vec3(0.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelaux));
Parabrisas_M.RenderModelWithTexture(&ojosTexture);

// — FAROS —————
modelaux = carBase;
modelaux = glm::translate(modelaux, glm::vec3(0.0f, 0.0f, 0.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelaux));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
Faros_M.RenderModel();

```

```

// — COFRE —————
modelaux = carBase;
modelaux = glm::translate(modelaux, glm::vec3(0.0f, 0.5f, 1.8f));
modelaux = glm::rotate(modelaux, hoodAngle * toRadians, glm::vec3(1.0f, 0.0f, 0.0f));
modelaux = glm::translate(modelaux, glm::vec3(0.0f, 0.0f, -0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelaux));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
Cofre_M.RenderModel();

// — LLANTAS (caucho) —————
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));

// Delantera Derecha
modelaux = carBase;
modelaux = glm::translate(modelaux, glm::vec3(1.3f, -0.55f, 1.4f));
modelaux = glm::rotate(modelaux, glm::radians(wheelAngle), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelaux));
LlantaDD_M.RenderModelWithTexture(&cauchoTexture);

// Trasera Derecha
modelaux = carBase;
modelaux = glm::translate(modelaux, glm::vec3(1.3f, -0.55f, -1.4f));
modelaux = glm::rotate(modelaux, glm::radians(wheelAngle), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelaux));
LlantaDT_M.RenderModelWithTexture(&cauchoTexture);

```

```

// Delantera Izquierda
modelaux = carBase;
modelaux = glm::translate(modelaux, glm::vec3(-1.3f, -0.55f, 1.4f));
modelaux = glm::rotate(modelaux, glm::radians(wheelAngle), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelaux));
LlantaID_M.RenderModelWithTexture(&cauchoTexture);

// Trasera Izquierda
modelaux = carBase;
modelaux = glm::translate(modelaux, glm::vec3(-1.3f, -0.55f, -1.4f));
modelaux = glm::rotate(modelaux, glm::radians(wheelAngle), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelaux));
LlantaIT_M.RenderModelWithTexture(&cauchoTexture);

// — RINES (metal) —————
// Rin Delantera Derecha
modelaux = carBase;
modelaux = glm::translate(modelaux, glm::vec3(1.3f, -0.55f, 1.4f));
modelaux = glm::rotate(modelaux, glm::radians(wheelAngle), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelaux));
RinDD_M.RenderModelWithTexture(&metalTexture);

// Rin Trasera Derecha
modelaux = carBase;
modelaux = glm::translate(modelaux, glm::vec3(1.3f, -0.55f, -1.4f));
modelaux = glm::rotate(modelaux, glm::radians(wheelAngle), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelaux));
RinDT_M.RenderModelWithTexture(&metalTexture);

```

```

// Rin Trasera Derecha
modelaux = carBase;
modelaux = glm::translate(modelaux, glm::vec3(1.3f, -0.55f, -1.4f));
modelaux = glm::rotate(modelaux, glm::radians(wheelAngle), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelaux));
RinDT_M.RenderModelWithTexture(&metalTexture);

// Rin Delantera Izquierda
modelaux = carBase;
modelaux = glm::translate(modelaux, glm::vec3(-1.3f, -0.55f, 1.4f));
modelaux = glm::rotate(modelaux, glm::radians(wheelAngle), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelaux));
RinID_M.RenderModelWithTexture(&metalTexture);

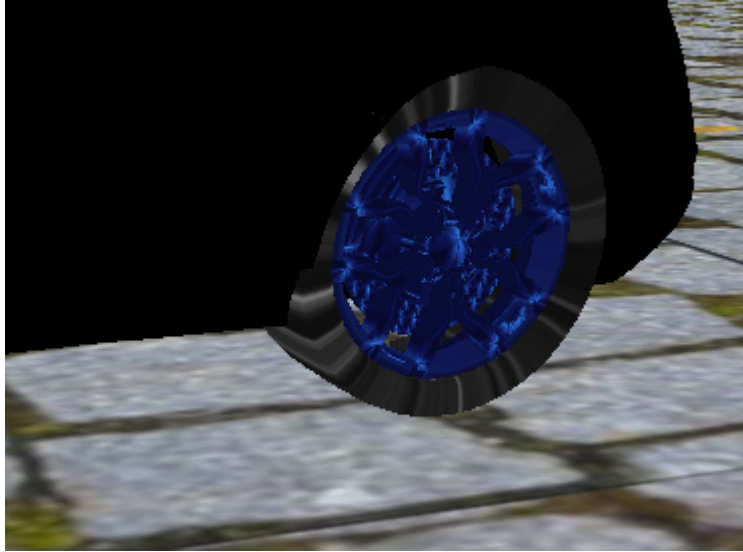
// Rin Trasera Izquierda
modelaux = carBase;
modelaux = glm::translate(modelaux, glm::vec3(-1.3f, -0.55f, -1.4f));
modelaux = glm::rotate(modelaux, glm::radians(wheelAngle), glm::vec3(0.0f, 0.0f, 1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelaux));
RinIT_M.RenderModelWithTexture(&metalTexture);

glUseProgram(0);
mainWindow.swapBuffers();
}

```

**Como en la práctica anterior vimos jerarquía ocupamos las mismas base, solo agregamos los nuevos modelos, además de darle una textura los rines (Metálica Azul) Llantas (Las figuras de llantas) y Parabrisas (Los ojos).

Ejecución de las llantas y rines



Los cambios de que realice fueron model.ccp

Lo que hace es que yo pueda elegir mis propias texturas, y no es escriban al importar en blender.

```
// Renderiza el modelo con sus propias texturas (.mtl)
void Model::RenderModel()
{
    for (unsigned int i = 0; i < MeshList.size(); i++)
    {
        unsigned int materialIndex = meshTotex[i];
        if (materialIndex < TextureList.size() && TextureList[materialIndex])
        {
            TextureList[materialIndex]->UseTexture();
        }
        MeshList[i]->RenderMesh();
    }
}

// Renderiza el modelo forzando una textura externa (PNG)
// Úsala cuando quieras aplicar tu propia textura en lugar de la del .mtl
void Model::RenderModelWithTexture(Texture* texture)
{
    for (unsigned int i = 0; i < MeshList.size(); i++)
    {
        texture->UseTexture();
        MeshList[i]->RenderMesh();
    }
}
```

El segundo cambio lo realice para según yo aparecieran los ojos pero no lo logre.

```

1  #version 330
2  in vec4 vColor;
3  in vec2 TexCoord;
4
5  out vec4 color;
6  uniform sampler2D theTexture;
7
8  void main()
9  {
10
11      color = texture(theTexture, TexCoord);
12      color = vColor*color;
13      //color = vColor*texture(theTexture, TexCoord);
14
15  }

```

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla.

1. Tuve problemas al cargar la textura de los ojos logrando que apareciera en ejecución.
2. Me costó mucho la parte de mapeo en GIMP, porque salieron volteadas las coordenadas. Casi me vuelvo loca.
3. Además que me parecían los texturizados correctos.
4. También tuve que modificar algunas cosas para que cargaran los texturizados en este punto.

3.- Conclusión

La parte de texturizado me costó mucho trabajo, porque las coordenadas no eran las correctas y eso hacía que aparecieran mal los dos cubos, en el carro logre que el rines tuviera un aspecto metálico, las llantas tiene como textura una llanta.

4.-Bibliografía en formato APA

Roque, J. (2026). *opengl3.3ydosmain* [Material de código]. UMAN.

